KI-Experiment: App aus der Anforderungsspezifikation (R13)

Optionale Aufgabe aus dem Merkblatt (S. 1, Checkliste M8): «aus der Anforderungsspezifikation mit KI-Agenten eine Applikation erstellen und analysieren, ob der Agent alle nötigen Requirements hat». Durchgeführt am 25.10.2026 mit der Spezifikation **v0.5** (Stand vor dem Interview in PVA 5).

Datei	Inhalt
`analyse.md`	**Auswertung:** Abdeckung je Anforderung, Annahmen des Agenten als Lücken der Spezifikation, Fazit, Änderungen für v0.6, Lessons Learned
`app/`	die App, wie der Agent sie abgegeben hat (unverändert), mit seiner `README.md` und `ANNAHMEN.md`
`agent-protokoll.md`	alle Werkzeugaufrufe des Agenten mit Zeit, sein Abschlussbericht wörtlich
`screenshots/`	26 Bildschirmfotos unseres Testlaufs
`test/`	unser Testskript (Playwright) und das Ergebnis des Laufs

Ziel

Wir wollten wissen, ob unsere Spezifikation für sich allein verständlich und vollständig genug ist, dass jemand ohne unser Hintergrundwissen daraus ein System bauen kann. Ein KI-Agent eignet sich als Testleser, weil er nur hat, was im Dokument steht, und weil er jede Stelle, an der er raten muss, protokollieren kann. **Jede geratene Stelle ist eine Lücke oder Mehrdeutigkeit der Spezifikation.**

Die App selbst ist kein Lösungsvorschlag für ArenaOne. Make-or-Buy entscheidet Gruppe 2 (RB-03).

Agent und Modell

Werkzeug	Claude Code (Anthropic), Agent-Werkzeug: ein neu gestarteter Subagent vom Typ «general-purpose»
Modell	Claude Opus 5.5 (`claude-opus-5-5`)
Umgebung	Linux-Rechner mit Node.js 22 und Python 3; Werkzeuge: Dateien lesen/schreiben, Shell. Internetzugang vorhanden (der Agent hat ihn nur genutzt, um Prüfwerkzeuge für seinen eigenen Browsertest zu laden).
Datum	25.10.2026, 14:52–15:20 UTC (15:52–16:20 Uhr Zürich)

Wer den Versuch aufgesetzt und ausgewertet hat: ebenfalls ein KI-Assistent (Claude Code) im Auftrag von Görkem Taspinar. Die Auswertung (`analyse.md`) stützt sich auf Testlauf, Logiktests und Quelltext der App; die Bewertungen sind ein Vorschlag, den Görkem Taspinar und Ardian Ibrahimaj vor der Präsentation gegenlesen.

Was der Agent bekam

- **Nur** die Datei `anforderungsspezifikation-arenaone.md` v0.5 (identisch mit dem Stand auf `main` nach PR #52, SHA-256 `0b85db8a…5740`), kopiert in einen leeren Arbeitsordner ausserhalb des Repos.
- Keine Bilder (Kontext- und Use-Case-Diagramm sind in der Markdown-Datei nur verlinkt und lagen nicht bei), keine anderen Dokumente aus `re-gruppe2/` (Analyse, Glossar, simuliertes Interview), keine Präsentation von Gruppe 2, kein Gespräch.
- Der Subagent startet ohne den Verlauf unserer Sitzung. Das Protokoll (`agent-protokoll.md`) zeigt, dass er ausser der Spezifikation keine Datei gelesen hat.
- Rückfragen waren nicht möglich: ein Auftrag, eine Abgabe.

Auftrag an den Agenten (wörtlich)

Pfade gekürzt: `<scratchpad>` steht für das temporäre Verzeichnis der Sitzung.

```
Du bekommst eine Anforderungsspezifikation und sollst daraus eine Applikation bauen.

Spezifikation: <scratchpad>/ki-versuch/input/anforderungsspezifikation-arenaone.md
Arbeitsordner (nur hier schreiben): <scratchpad>/ki-versuch/app/

Auftrag:
- Baue aus der Spezifikation eine lauffähige Web-App.
- Form: statische Web-App (HTML, CSS, JavaScript) ohne Build-Schritt und ohne Server, sodass
sie direkt auf GitHub Pages läuft. Daten im Browser (localStorage), mit Demo-Daten beim
ersten Start.
- Wo die Spezifikation unklar, widersprüchlich oder lückenhaft ist, triff eine Annahme und
protokolliere sie in ANNAHMEN.md (je Annahme: betroffene Stelle/ID in der Spezifikation, was
unklar war, was du angenommen hast). Fragen, die du einer Auftraggeberin stellen würdest,
notierst du ebenfalls dort.
- Lege eine kurze README.md in den Arbeitsordner (wie starten, was umgesetzt ist).
- Verwende ausschliesslich die Spezifikation als Grundlage. Lies keine anderen Dateien
ausserhalb des Arbeitsordners und der Spezifikation und such nicht im Dateisystem nach
weiterem Material.
- Externe Bibliotheken nur, wenn nötig, und dann lokal im Ordner oder über ein CDN.

Antworte am Ende mit einer kurzen Zusammenfassung: was gebaut wurde, welche Teile der
Spezifikation du nicht umgesetzt hast und warum, und wie viele Annahmen du protokolliert
hast.
```

Die Zielform (statisch, localStorage) haben wir vorgegeben, damit die App ohne Server auf GitHub Pages läuft. Das ist eine Einschränkung des Versuchs, nicht der Spezifikation: Anforderungen an Betrieb, Last und gemeinsame Daten (QA-05, QA-07 bis QA-09, QA-16) kann eine solche App nicht erfüllen. In der Analyse sind sie deshalb getrennt ausgewiesen.

Vorgehen

1. Spezifikation in einen leeren Ordner kopiert, Prüfsumme festgehalten.
2. Subagent mit dem Auftrag oben gestartet und bis zur Abgabe nicht eingegriffen.
3. Ergebnis unverändert nach `app/` kopiert (Vergleich mit `diff -r`: identisch), Protokoll und Abschlussbericht gesichert.

4. Die Logiktests des Agenten laufen lassen (`node app/tests/logik-test.js`: 28 bestanden, 0 fehlgeschlagen).
5. Eigener Testlauf mit Playwright und Chromium entlang der Hauptabläufe, Uhr fest auf Montag, 26.10.2026, 08:00 Uhr, damit die Demo-Daten reproduzierbar sind (`test/szenario.js`, Ergebnis `test/testlauf-ergebnis.txt`: 19 Abläufe ohne Fehler, keine JavaScript-Fehler).
6. Jede Anforderung (35 FA, 16 QA, 9 Use Cases) gegen App, Tests und Screenshots bewertet; die 32 Annahmen und 15 Fragen des Agenten den Kapiteln der Spezifikation zugeordnet (`analyse.md`).
7. Befunde in die Spezifikation übernommen (v0.6: Kapitel 10.2, 11.3, 13).

Zeitaufwand

Gemessen an den Zeitstempeln der Sitzung (25.10.2026, UTC). Aufsetzen, Testlauf und Auswertung hat ebenfalls ein KI-Assistent erledigt; für Menschen wären diese Schritte deutlich länger.

Schritt	Zeit (UTC)	Dauer
Versuch aufsetzen (Spezifikation kopieren, Auftrag formulieren)	14:47–14:52	ca. 5 min
Agent baut die App, ohne Eingriff	14:52–15:20	28 min (35 Werkzeugaufrufe, rund 290 000 Tokens)
Testlauf mit Screenshots (Skript schreiben, Selektoren anpassen)	15:20–15:35	ca. 15 min
Auswertung, Spezifikation v0.6, Folien, PDFs	15:35–15:50	ca. 15 min, teils parallel zum Testlauf vorbereitet
Gegenlesen der Auswertung im Team	offen	geschätzt 30–45 min

Zum Vergleich: Die Spezifikation v0.5 selbst hat ein Mehrfaches davon gekostet. Der Agent ersetzt die Ermittlung nicht, er prüft nur, ob das Ergebnis lesbar und eindeutig ist.

App starten

`app/index.html` im Browser öffnen (auch direkt als Datei). Anmeldung und Demo-Konten stehen auf der Anmeldeseite; Kundschaft `anna` / `demo1234`, Mitarbeitende `empfang`, `gl`, `admin` / `arenaone-demo` mit simuliertem zweitem Faktor. Alle Daten bleiben im Browser.

Testlauf wiederholen (Node.js und Playwright, Chromium):

```
APP=$PWD/app OUT=$PWD/screenshots node test/szenario.js
```

`test/h.js` erwartet Chromium unter `~/.cache/ms-playwright/chromium-1248/`; auf einem anderen Rechner dort den Pfad anpassen.

KI-Deklaration

In diesem Teil ist KI der Gegenstand des Versuchs: Die App unter `app/` ist vollständig von einem KI-Agenten geschrieben, ohne Änderung durch uns. Auch Testskript, Auswertung und die Änderungen an der Spezifikation sind mit KI-Unterstützung (Claude Code) entstanden; das Team liest sie vor der Abgabe gegen. Ob und in welcher Form die HWZ eine Deklaration verlangt, ist noch offen (FAHRPLAN O10).